```
import Data.Char(toLower)
import Data.List(sort, group, genericLength)
import Test.Hspec
    ( shouldBe, it, describe, hspec, Spec, errorCall, shouldThrow )
import Test.QuickCheck ()
import Control.Exception (evaluate)


-- Identificação:   Matheus José da Costa | 11711BCC008

-------------------------------------------------------------------------------
-----------------------------------------------
--Questão 01
éPalíndromo:: Eq a => [a] -> Bool
éPalíndromo lista = reverse lista == lista

-------------------------------------------------------------------------------
-----------------------------------------------
--Questão 02

convertStringIntoList :: String -> [String]
convertStringIntoList "" = []
convertStringIntoList string = take 1 string : convertStringIntoList (drop 1
string)

lowerString :: String -> String
lowerString string = [ toLower loweredString | loweredString <- string]

removeDuplicatas :: Eq a => [a] -> [a]
removeDuplicatas [] = []
removeDuplicatas (x:xs)
   | x `elem` xs = removeDuplicatas xs
   | otherwise = x:removeDuplicatas xs

removeUnusedElements :: [String] -> [String]
removeUnusedElements [" "] = []
removeUnusedElements [","] = []
removeUnusedElements [";"] = []
removeUnusedElements ["."] = []
removeUnusedElements ["-"] = []
removeUnusedElements ["_"] = []
removeUnusedElements ["%"] = []
removeUnusedElements ["$"] = []
removeUnusedElements ["#"] = []
removeUnusedElements ["@"] = []
removeUnusedElements ["!"] = []
removeUnusedElements ["&"] = []
removeUnusedElements ["*"] = []
removeUnusedElements ["("] = []
removeUnusedElements [")"] = []
removeUnusedElements ["+"] = []
removeUnusedElements [a] = [a]
removeUnusedElements (x:xs) = if x `elem` alfabeto then x:removeUnusedElements
xs else removeUnusedElements xs
    where
         alfabeto = convertStringIntoList ['a'..'z']

éPangrama :: String -> Bool
éPangrama string = resultado
    where
        alfabeto = convertStringIntoList ['a'..'z']
        stringCorreta = removeDuplicatas (removeUnusedElements
(convertStringIntoList (sort (lowerString string))))
        resultado = and(zipWith (==) alfabeto stringCorreta)
```

```
-------------------------------------------------------------------------------
-----------------------------------------
--Questão 03

data Árvore arvore = Nada | Nó arvore (Árvore arvore) (Árvore arvore)  deriving
(Show)

adicionar :: (Eq a, Ord a) => a -> Árvore a -> Árvore a
adicionar novoDado Nada = Nó novoDado Nada Nada
adicionar novoDado árvore@(Nó dadoExistente ae ad)
    | novoDado == dadoExistente = árvore
    | novoDado < dadoExistente  = Nó dadoExistente (adicionar novoDado ae) ad
    | otherwise                 = Nó dadoExistente ad (adicionar novoDado ad)

impressãoEmOrdem :: (Show a) => Árvore a -> String
impressãoEmOrdem Nada = "."
impressãoEmOrdem (Nó dado ae ad) = impressãoEmOrdem ae ++ show dado ++
impressãoEmOrdem ad
-------------------------------------------------------------------------------
-----------------------------------------
--Questão 04

aplicação :: (t1 -> t2) -> t1 -> t2
aplicação funcao = funcao

aplicação2 :: (t -> t) -> t -> t
aplicação2 funcao number = funcao (funcao number)

aplicação3 :: (a -> a) -> a -> Int -> a
aplicação3 funcao number times = iterate funcao number !! times

-------------------------------------------------------------------------------
-----------------------------------------
--Questão 05

soma :: Num a => [a] -> a
soma [] = 0
soma (x:xs) = x+soma xs

média :: (Fractional p, Real a) => [a] -> p
média [] = error "Nao existe media para uma operacao vazia"
média lista = realToFrac (soma lista) / genericLength lista

médias :: (Fractional b, Real a) => [[a]] -> [b]
médias [] = error "Nao existe media para uma operacao vazia"
médias listas = map média listas
-------------------------------------------------------------------------------
-----------------------------------------
--Questão 06

-------------------------------------------------------------------------------
-----------------------------------------
--Questão 07
prepareList :: Ord a => [a] -> [[a]]
prepareList list = group (sort list)

createTupleWithElementAndSize :: [[a]] -> [(a, Int)]
createTupleWithElementAndSize = map (\ x -> (head x, length x))

countElementsFromAList :: Ord a => [a] -> [(a, Int)]
countElementsFromAList lista = resultado
    where
        lista1 = prepareList lista
        resultado = createTupleWithElementAndSize lista1
```

```
main :: IO ()
main = hspec spec

spec::Spec
spec = do
        describe "Questão 01" $ do
                it "Teste palíndromo 1" $ do
                        éPalíndromo [1,2,3,2,1] `shouldBe` True
                it "Teste palíndromo 2" $ do
                        éPalíndromo "abba" `shouldBe` True
                it "Teste palíndromo 3" $ do
                        éPalíndromo "abBA" `shouldBe` False
                it "Teste palíndromo 4" $ do
                        éPalíndromo [1.0,1.0,2.0] `shouldBe` False
        describe "Questão 02" $ do
                it "Pangrama 1" $ do
                        éPangrama "Bancos fúteis pagavam-lhe queijo, whisky e
xadrez" `shouldBe` True
                it "Pangrama 2" $ do
                        éPangrama "Já fiz vinho com toque de kiwi para belga
sexy" `shouldBe` True
                it "Pangrama 3" $ do
                        éPangrama "The quick brown fox jumps over the lazy dog"
`shouldBe` True
                it "Pangrama 4" $ do
                        éPangrama "Huguinho, Zézinho e Luisinho" `shouldBe`
False
                it "Pangrama 5" $ do
                        éPangrama "The quick brown.Fox jumps over the lazy dog.!
@#$%&*()+" `shouldBe` True
        describe "Questão 04" $ do
                it "Aplicação 1 - Teste 01" $ do
                        aplicação (^2) 3 `shouldBe` 9
                it "Aplicação 1 - Teste 02" $ do
                        aplicação (*2) 3 `shouldBe` 6
                it "Aplicação 2 - Teste 01" $ do
                        aplicação2 (^2) 3 `shouldBe` 81
                it "Aplicação 2 - Teste 02" $ do
                        aplicação2 (^^2) 3 `shouldBe` 81.0
                it "Aplicação 3 - Teste 01" $ do
                        aplicação3 (^2) 3 2 `shouldBe` 81
                it "Aplicação 3 - Teste 02" $ do
                        aplicação3 (+1) 3 4 `shouldBe` 7.0
        describe "Questão 05" $ do
                it "Média - Teste 01" $ do
                        média [1,2,3] `shouldBe` 2
                it "Média - Teste 02" $ do
                        média [1,2,3,4,5] `shouldBe` 3
                it "Média - Teste 03" $ do
                        evaluate(média []) `shouldThrow` errorCall "Nao existe
media para uma operacao vazia"
                it "Médias - Teste 01" $ do
                        médias [[1,2,3],[4,5,6]] `shouldBe` [2,5]
                it "Médias - Teste 02" $ do
                        médias [[1,2,3],[4,5,6]] `shouldBe` [2,5]
                it "Médias - Teste 03" $ do
                        evaluate(médias []) `shouldThrow` errorCall "Nao existe
media para uma operacao vazia"

        describe "Questão 05" $ do
                it "Média - Teste 01" $ do
                        média [1,2,3] `shouldBe` 2
                it "Média - Teste 02" $ do
```

```
                        média [1,2,3,4,5] `shouldBe` 3
                it "Médias - Teste 01" $ do
                        médias [[1,2,3],[4,5,6]] `shouldBe` [2,5]
                it "Médias - Teste 02" $ do
                        médias [[1,2,3],[4,5,6]] `shouldBe` [2,5]
        describe "Questão 07" $ do
                it "Conta elementos de uma lista - Teste 01" $ do
                        countElementsFromAList [1,2,3] `shouldBe` [(1,1),(2,1),
(3,1)]
                it "Conta elementos de uma lista - Teste 02" $ do
                        countElementsFromAList [1,2,3,4,5,1,2] `shouldBe`
[(1,2),(2,2),(3,1),(4,1),(5,1)]
```